Week 4 Level 3: Advanced

Searching and How Fast Is Fast?

Smart algorithms do less work: halving beats checking one by one.

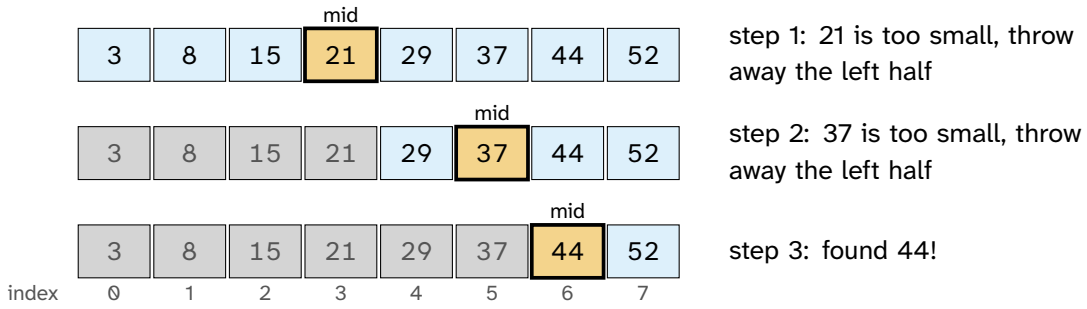
What you will learn

- ☐ Search a list in two ways: linear search and binary search.
- ☐ Count the steps an algorithm takes, and compare how they grow.
- ☐ Build a bot that always guesses your number in 7 tries or fewer.

Two ways to search

Linear search looks at the items one by one, from the start. It works on any list, but a long list means a long search.

Binary search only works on a **sorted** list. It looks at the middle item. If that is too small, the answer must be on the right, so it throws away the whole left half (and the other way round). Every step cuts the list in half.



Gray boxes have been thrown away; the thick orange box is the one we look at (`mid`). Searching for 44 took 3 steps. Linear search would take 7.

How fast is fast?

To compare algorithms, count the **steps** in the **worst case** (the unluckiest search).

Items in the list	Linear search	Binary search
10	10 steps	4 steps
100	100 steps	7 steps
1000	1000 steps	10 steps

When the list gets 10 times bigger, linear search does 10 times more work. Binary search only does about 3 more steps!

Binary search in Python

```python
low = 0
high = len(items) - 1          # the LAST index, not the length
while low <= high:             # <= : one item left is still worth
    ↪ checking
    mid = (low + high) // 2    # // makes a whole number
    if items[mid] == target:
        return mid
    elif items[mid] < target:
        low = mid + 1          # throw away the left half (and mid)
    else:
        high = mid - 1         # throw away the right half (and mid)
return -1                      # nothing left: not in the list
```

Exercise 4.1: Search Race

Folder: `week04/ex-search-race`. `linear_search` is done. Write `binary_search`: it returns `(index, steps)`. Run it to see the race table.
Predict the binary steps for 10000 items: _____ What did the program say? _____

Exercise 4.2: Guess-the-Number Bot

Folder: `week04/ex-guess-bot`. First run the slow bot: it guesses 1, 2, 3... (press the red Stop button). Then:

- finish `check(guess, secret)`: return `"higher"`, `"lower"` or `"correct"`;
- make `next_guess(low, high)` return the middle.

Play 3 games. How many guesses did the bot need? _____ _____ _____

Tip: Filter bubbles

Search engines are amazingly fast, but they also choose which results to show you first, based on what you clicked before. If you only ever see what you already like, you are in a “filter bubble”. Try different search words, look at more than one source, and ask a grown-up when something seems odd.

Vocabulary box

algorithm A clear list of steps that solves a problem.

linear search Check items one by one from the start.

binary search In a sorted list, check the middle and throw away the half that cannot hold the answer.

worst case The unluckiest situation, when an algorithm needs the most steps.

off-by-one error A bug where a number is 1 too big or 1 too small, like using < instead of <=.

Reflection

Explain to a younger friend why binary search needs a sorted list:

Where in real life do you use halving to find something?

Week 4 Level 3: Advanced

Searching and How Fast Is Fast?

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Growth Table (about 10 minutes)

1. Predict first. How many steps does each search need in the worst case? Fill in the empty boxes *before* you run any code.

Items	Linear	Binary
10		
100		
1000		
1 000 000		

2. Code. Open `week04/hw-growth-table/starter.py`. Finish `linear_worst(n)` and `halving_steps(n)`: keep doing `n = n // 2` and count, until n is 0.

3. Check. Run it and compare with your predictions. Then open `check_my_work.py` and press Run. Commit your work in `git-quest`.

★★ Challenge (want more?): Trace it by hand

Use binary search to find **9** in `[1, 3, 5, 7, 9, 11, 13]`. Fill in one row per step.

step	low	high	mid	items[mid]	so...
1					
2					
3					

How many steps did it take? _____ How many would linear search take? _____

🎨 Creative (make it yours): Draw the race

Draw a bar chart of your growth table on paper or with turtle: one pair of bars for each list size. Linear bars get too tall very quickly! How could you draw them so they still fit on the page? (Idea: let 1 cm stand for more and more steps, and write the real number on each bar.)

Self-check: tick each box when it is true

- ☐ I wrote my predictions before running the code.
- ☐ All checks pass in `hw-growth-table`.
- ☐ I can explain why binary search grows so slowly.
- ☐ I committed my work in `git-quest`.